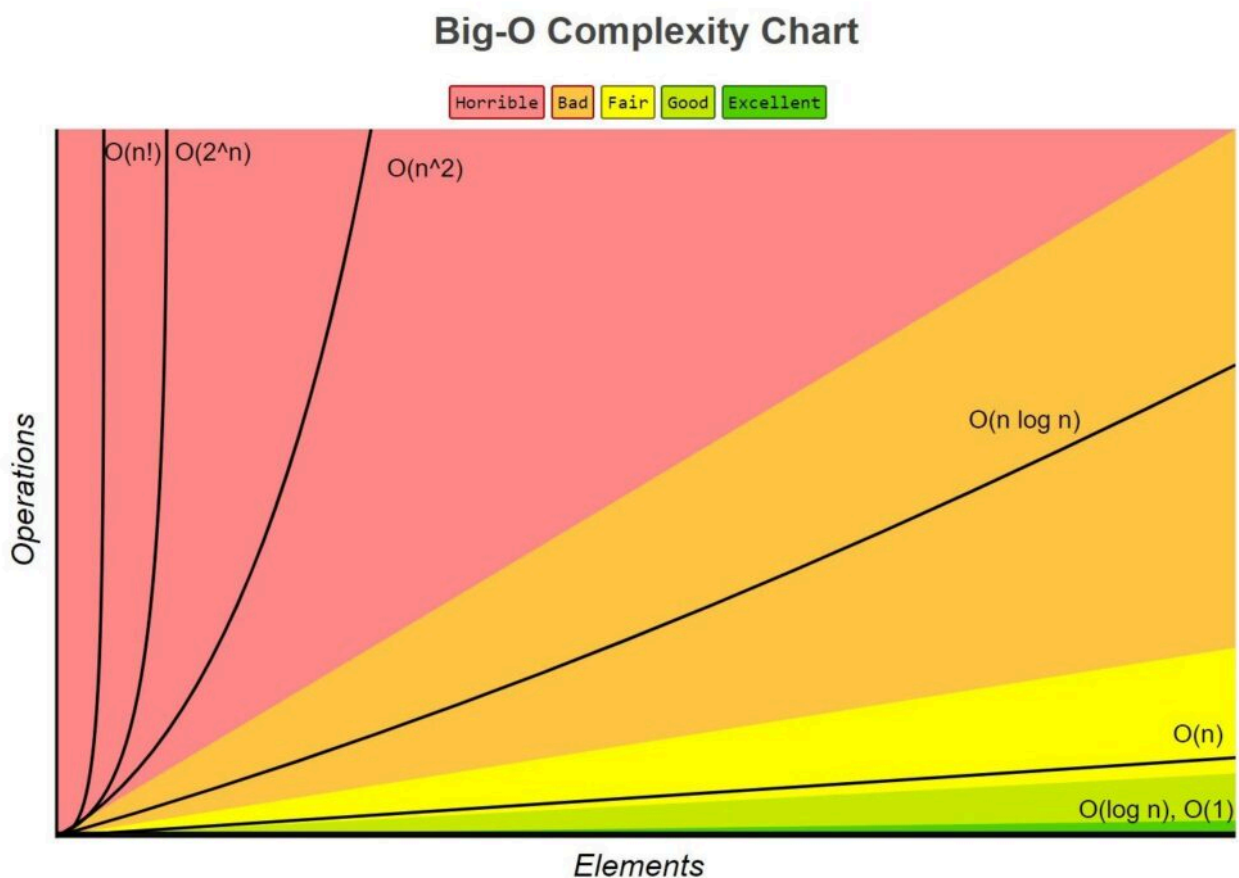


22 – Big Oh and Theta

Algunas Clases de Complejidad Importantes

Algunas de las instancias más comunes de Big O (y Θ) se listan abajo. En cada caso, n es una medida del tamaño de las entradas a la función.

- $O(1)$ denota tiempo de ejecución constante.
- $O(\log n)$ denota tiempo de ejecución logarítmico.
- $O(n)$ denota tiempo de ejecución lineal.
- $O(n \log n)$ denota tiempo de ejecución log-lineal.
- $O(n^k)$ denota tiempo de ejecución polinomial. Nota que k es una constante.
- $O(c^n)$ denota tiempo de ejecución exponencial. Aquí una constante está siendo elevada a una potencia basada en el tamaño de la entrada.



Complejidad Constante

Esto indica que la complejidad asintótica es independiente del tamaño de las entradas. Hay muy pocos programas interesantes en esta clase, pero todos los programas tienen piezas (por ejemplo, obtener la longitud de una lista de Python o multiplicar dos números de punto flotante) que encajan en esta clase. El tiempo de ejecución constante no implica que no hay bucles o llamadas recursivas en el código, pero sí implica que el número de iteraciones o llamadas recursivas es independiente del tamaño de las entradas.

Por ejemplo:

```
1 def ejemplo():
2     for i in range(5): # siempre 5 iteraciones
3         print(i)
```

En este código $O(1)$ tenemos un bucle pero este siempre va a realizar 5 pasos.

Complejidad Logarítmica

Tales funciones tienen una complejidad que crece como el logaritmo de al menos una de las entradas. La búsqueda binaria, por ejemplo, es logarítmica en la longitud de la lista siendo buscada. Por cierto, no nos importa la base del logaritmo, ya que la diferencia entre usar una base y otra es meramente un factor multiplicativo constante. Por ejemplo, $O(\log_2(10)\log_2(x)) = O(\log_{10}(x))$. Hay muchas funciones interesantes con complejidad logarítmica. Considera

```
1 def int_to_str(i):
2     """Asume que i es un int no negativo
3     Retorna una representación de cadena decimal de i"""
4     digits = '0123456789'
5     if i == 0:
6         return '0'
7     result = ''
8     while i > 0:
9         result = digits[i % 10] + result
10        i = i // 10
11    return result
```

Como no hay llamadas a funciones o métodos en este código, sabemos que solo tenemos que mirar los bucles para determinar la clase de complejidad. Hay solo un bucle, así que lo único que necesitamos hacer es caracterizar el número de iteraciones. Eso se reduce al número de veces que podemos usar `//` (división entera) para dividir `i` por 10 antes de obtener como resultado de 0. Por lo tanto, la complejidad de

`int_to_str` es $O(\log(i))$. Más precisamente, es de orden $\theta(\log(i))$, porque $\log(i)$ es una cota ajustada.

Qué hay de la complejidad de:

```
1 def add_digits(n):
2     """Asume que n es un entero no negativo
3     Devuelve la suma de los dígitos en n"""
4     string_rep = int_to_str(n)
5     val = 0
6     for c in string_rep:
7         val += int(c)
8     return val
```

La complejidad de convertir n a una cadena usando `int_to_str` es de orden $\theta(\log(n))$, y `int_to_str` devuelve una cadena de longitud $\log(n)$. El bucle `for` se ejecutará orden $\theta(\text{len}(\text{string_rep}))$ veces, es decir, orden $\theta(\log(n))$ veces. Juntándolo todo, y asumiendo que un carácter que representa un dígito puede ser convertido a un entero en tiempo constante, el programa se ejecutará en tiempo proporcional a $\theta(\log(n)) + \theta(\log(n))$, lo cual lo convierte en orden $\theta(\log(n))$.

Complejidad Lineal

Muchos algoritmos que tratan con listas u otros tipos de secuencias son lineales porque tocan cada elemento de la secuencia un número constante (mayor que 0) de veces.

Considera, por ejemplo,

```
1 def add_digits(s):
2     """Asume que s es una cadena de dígitos
3     Devuelve la suma de los dígitos en s"""
4     val = 0
5     for c in s:
6         val += int(c)
7     return val
```

Esta función es lineal en la longitud de s , es decir, $\theta(\text{len}(s))$.

Por supuesto, un programa no necesita tener un bucle para tener complejidad lineal. Considera

```
1 def factorial(x):
2     """Assumes that x is a positive int
3     Returns x!"""
4     if x == 1:
5         return 1
```

```
6     else:
7         return x * factorial(x-1)
```

No hay bucles en este código, así que para analizar la complejidad necesitamos averiguar cuántas llamadas recursivas se realizan. La serie de llamadas es simplemente

```
1  factorial(x), factorial(x-1), factorial(x-2), ... ,
2  factorial(1)
```

La longitud de esta serie, y por tanto la complejidad de la función, es de orden $\theta(x)$.

Hasta ahora solo hemos examinado la **complejidad temporal** de nuestro código. Esto está bien para algoritmos que usan una cantidad constante de espacio, pero esta implementación del factorial no tiene esa propiedad. Cada llamada recursiva de **factorial** causa que se asigne un nuevo marco de pila, y ese marco continúa ocupando memoria hasta que la llamada retorna. En la máxima profundidad de recursión, este código habrá asignado x marcos de pila, por lo que la **complejidad espacial** también es $\theta(x)$.

El impacto de la complejidad espacial es más difícil de apreciar que el impacto de la complejidad temporal. Si un programa toma uno o dos minutos en completarse es bastante visible para su usuario, pero si usa uno o dos megabytes de memoria es en gran medida invisible para los usuarios. Esta es la razón por la que la gente típicamente presta más atención a la complejidad temporal que a la complejidad espacial. La excepción ocurre cuando un programa necesita más espacio del que está disponible en la memoria rápida de la máquina en la que se ejecuta.

Complejidad Log-Lineal

Esta es ligeramente más complicada que las clases de complejidad que hemos visto hasta ahora. Involucra el producto de dos términos, cada uno de los cuales depende del tamaño de las entradas. Es una clase importante, porque muchos algoritmos prácticos son log-lineales. El algoritmo log-lineal más comúnmente usado es probablemente **merge sort**, que es de orden $\theta(n * \log(n))$, donde n es la longitud de la lista que se está ordenando.

Por ejemplo:

Merge Sort Tipico

```

1  def merge_sort(lista):
2      if len(lista) <= 1:
3          return lista
4
5      mid = len(lista) // 2
6      left = merge_sort(lista[:mid])
7      right = merge_sort(lista[mid:])
8
9      return merge(left, right)
10
11 def merge(left, right):
12     resultado = []
13     i = j = 0
14     while i < len(left) and j < len(right):
15         if left[i] < right[j]:
16             resultado.append(left[i])
17             i += 1
18         else:
19             resultado.append(right[j])
20             j += 1
21     resultado.extend(left[i:])
22     resultado.extend(right[j:])
23     return resultado

```

El algoritmo divide los datos en partes mas pequeñas, y luego trabaja sobre todas ellas. En este caso el $\log n$ se da porque el array se divide recursivamente a la mitad `n -> n / 2 -> n / 4 -> ...` hasta llegar a 1. Y además en cada nivel hay que fusionar todas las partes, y fusionar cuesta $O(n)$ porque hay que recorrer todos los elementos. Por lo que la complejidad temporal es $O(n \log n)$.

Complejidad Polinomial

Los algoritmos polinomiales más comúnmente usados son cuadráticos, es decir, su complejidad crece como el cuadrado del tamaño de su entrada. Considera, por ejemplo, la siguiente función, que implementa una prueba de subconjunto.

Implementación de prueba de subconjunto

```

1  def is_subset(L1, L2):
2      """Asume que L1 y L2 son listas.
3      Devuelve True si cada elemento en L1 también está en L2
4      y False en caso contrario."""
5      for e1 in L1:
6          matched = False
7          for e2 in L2:
8              if e1 == e2:
9                  matched = True
10                 break

```

```

11         if not matched:
12             return False
13     return True

```

Cada vez que se alcanza el bucle interno se ejecuta orden $\theta(\text{len}(L2))$ veces. La función `is_subset` ejecutará el bucle externo orden $\theta(\text{len}(L1))$ veces, por lo que el bucle interno se alcanzará orden $\theta(\text{len}(L1) * \text{len}(L2))$ veces.

Ahora considera la función `intersect`. El tiempo de ejecución para la parte del código que construye la lista que podría contener duplicados es claramente de orden $\theta(\text{len}(L1) * \text{len}(L2))$. A primera vista, parece que la parte del código que construye la lista libre de duplicados es lineal en la longitud de `tmp`, pero no es así.

Implementación de intersección de listas

```

1  def intersect(L1, L2):
2      """Asume que L1 y L2 son listas
3      Devuelve una lista sin duplicados que es la intersección de
4      L1 y L2"""
5      #Construir una lista que contenga elementos comunes
6      tmp = []
7      for e1 in L1:
8          for e2 in L2:
9              if e1 == e2:
10                 tmp.append(e1)
11                 break
12      #Construir una lista sin duplicados
13      result = []
14      for e in tmp:
15          if e not in result:
16              result.append(e)
17      return result

```

Evaluar la expresión `e not in result` potencialmente involucra examinar cada elemento en `result`, y por tanto es $\theta(\text{len}(\text{result}))$; consecuentemente, la segunda parte de la implementación es de orden $\theta(\text{len}(\text{tmp}) * \text{len}(\text{result}))$. Sin embargo, dado que las longitudes de `result` y `tmp` están acotadas por la longitud de la más pequeña de `L1` y `L2`, y dado que ignoramos términos aditivos, la complejidad de `intersect` es $\theta(\text{len}(L1) * \text{len}(L2))$.

Complejidad Exponencial

Muchos problemas importantes son inherentemente exponenciales, es decir, resolverlos completamente puede requerir tiempo que es exponencial en el tamaño de la entrada. Esto es desafortunado, ya que

rara vez vale la pena escribir un programa que tenga una probabilidad razonablemente alta de tomar tiempo exponencial para ejecutarse. Considera, por ejemplo, el siguiente código.

Generando el conjunto potencia

```
1  def get_binary_rep(n, num_digits):
2      """Asume que n y numDigits son enteros no negativos
3      Devuelve un str de longitud numDigits que es una
4      representación binaria de n"""
5      result = ''
6      while n > 0:
7          result = str(n%2) + result
8          n = n//2
9      if len(result) > num_digits:
10         raise ValueError('not enough digits')
11     for i in range(num_digits - len(result)):
12         result = '0' + result
13     return result
14
15 def gen_powerset(L):
16     """Asume que L es una lista
17     Devuelve una lista de listas que contiene todas las posibles
18     combinaciones de los elementos de L. Por ejemplo, si
19     L es [1, 2] devolverá una lista con elementos
20     [], [1], [2], y [1,2]."""
21     powerset = []
22     for i in range(0, 2**len(L)):
23         bin_str = get_binary_rep(i, len(L))
24         subset = []
25         for j in range(len(L)):
26             if bin_str[j] == '1':
27                 subset.append(L[j])
28         powerset.append(subset)
29     return powerset
```

La función `gen_powerset(L)` devuelve una lista de listas que contiene todas las posibles combinaciones de los elementos de L. Por ejemplo, si L es `['x', 'y']`, el conjunto potencia de L será una lista que contiene las listas `[]`, `['x']`, `['y']`, y `['x', 'y']`.

El algoritmo es un poco sutil. Considera una lista de n elementos. Podemos representar cualquier combinación de elementos por una cadena de n 0's y 1's, donde un 1 representa la presencia de un elemento y un 0 su ausencia. La combinación que no contiene elementos está representada por una cadena de todos 0's, la combinación que contiene todos los elementos está representada por una cadena de todos 1's, la combinación que contiene solo el primer y último elementos está representada por 100 ... 001, etc.

Generar todos los subconjuntos de una lista L de longitud n puede hacerse de la siguiente manera:

- Generar todos los números binarios de n bits. Estos son los números de 0 a $2^n - 1$.
- Para cada uno de estos 2^n números binarios, b , generar una lista seleccionando aquellos elementos de L que tienen un índice correspondiente a un 1 en b . Por ejemplo, si L es `['x', 'y', 'z']` y b es 101, generar la lista `['x', 'z']`.

Intenta ejecutar `gen_powerset` en una lista que contenga las primeras 10 letras del alfabeto. Terminará bastante rápido y producirá una lista con 1024 elementos. Luego, intenta ejecutar `gen_powerset` en las primeras 20 letras del alfabeto. Tomará más que un poco de tiempo para ejecutarse, y devolverá una lista con cerca de un millón de elementos. Si intentas ejecutar `gen_powerset` en las 26 letras completas, probablemente te cansarás de esperar que termine, a menos que tu computadora tenga mucha memoria para construir una lista con decenas de millones de elementos. Ni siquiera pienses en intentar ejecutar `gen_powerset` en una lista que contenga todas las letras mayúsculas y minúsculas. El paso 1 del algoritmo genera orden $\theta(2^{\text{len}(L)})$ números binarios, por lo que el algoritmo es exponencial en $\text{len}(L)$.

¿Significa esto que no podemos usar la computación para abordar problemas exponencialmente difíciles? Absolutamente no. Significa que tenemos que encontrar algoritmos que proporcionen soluciones aproximadas a estos problemas o que encuentren soluciones exactas en algunas instancias del problema.

Comparaciones de Clases de Complejidad

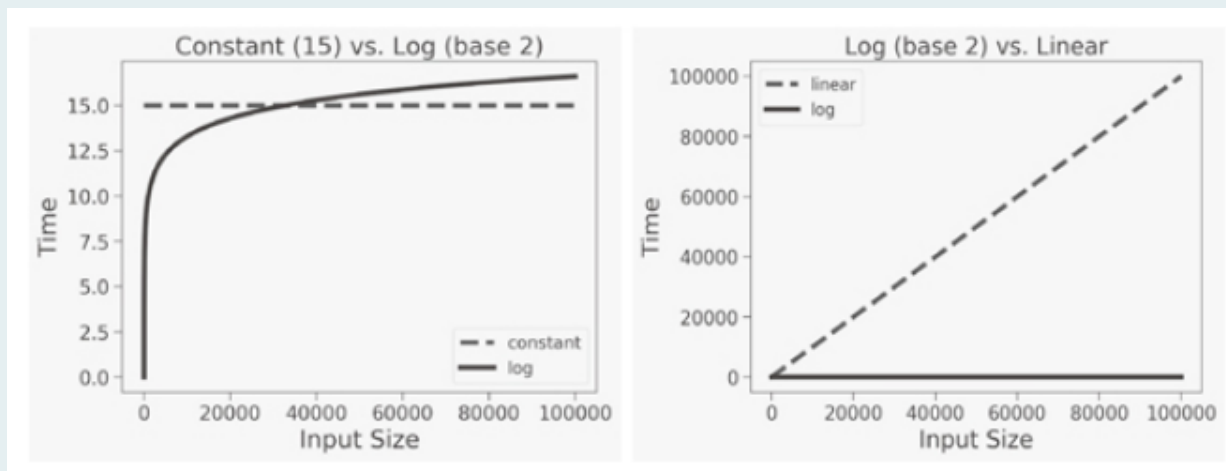
Los gráficos en esta sección están destinados a transmitir una impresión de las implicaciones de que un algoritmo esté en una u otra de estas clases de complejidad.

El gráfico de la izquierda compara el crecimiento de un algoritmo de tiempo constante con el de un algoritmo logarítmico. Nota que el tamaño de la entrada tiene que llegar a cerca de 30,000 para que los dos se crucen, incluso para la constante muy pequeña de 15. Cuando el tamaño de la entrada es 100,000, el tiempo requerido por un algoritmo logarítmico sigue siendo bastante pequeño. La moraleja es que los algoritmos logarítmicos son casi tan buenos como los de tiempo constante.

El gráfico de la derecha ilustra la diferencia dramática entre algoritmos logarítmicos y lineales.

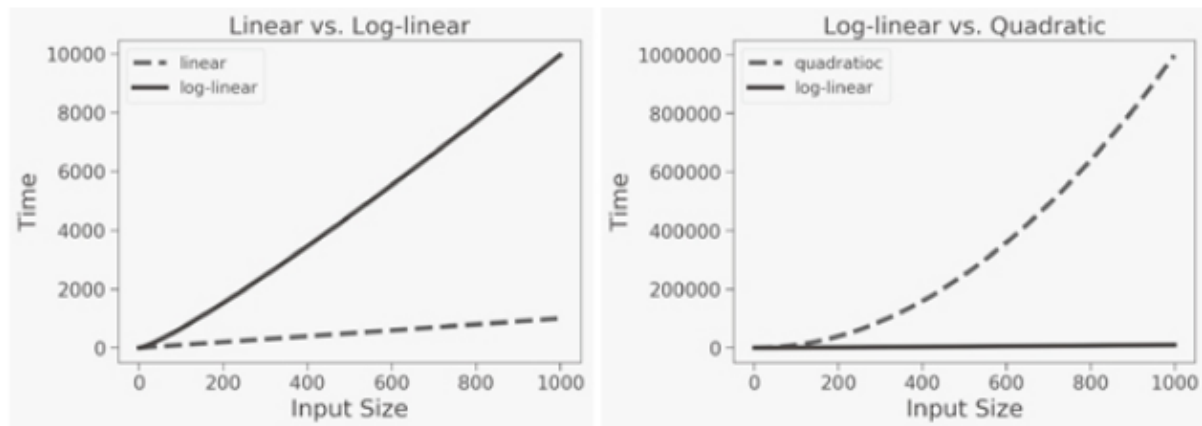
Mientras que necesitamos observar entradas grandes para apreciar la diferencia entre algoritmos de tiempo constante y logarítmico, la diferencia entre algoritmos logarítmicos y lineales es evidente incluso en entradas pequeñas. La diferencia dramática en el rendimiento relativo de algoritmos logarítmicos y lineales no significa que los algoritmos lineales sean malos. De hecho, la mayoría de las veces un algoritmo lineal es aceptablemente eficiente.

❶ Crecimiento constante, logarítmico y lineal



El gráfico de la izquierda muestra que hay una diferencia significativa entre $O(n)$ y $O(n \log(n))$. Dado lo lentamente que crece $\log(n)$, esto puede parecer sorprendente; pero ten en cuenta que es un factor multiplicativo. También ten en cuenta que en muchas situaciones prácticas, $O(n \log(n))$ es lo suficientemente rápido para ser útil. Por otro lado, como sugiere el gráfico de la derecha, hay muchas situaciones en las que una tasa de crecimiento cuadrática es prohibitiva.

❷ Crecimiento lineal, log-lineal y cuadrático



[Figure 11-8](#) Linear, log-linear, and quadratic growth

Los gráficos tratan sobre la complejidad exponencial. En el gráfico de la izquierda, los números a la izquierda del eje y van de 0 a 6. Sin embargo, la notación $1e29$ en la parte superior izquierda significa que cada marca en el eje y debe multiplicarse por 10^{29} . Por lo tanto, los valores y graficados van desde 0 hasta aproximadamente $6.6 * 10^{29}$. La curva para el crecimiento cuadrático es casi invisible en el gráfico de la izquierda. Eso es porque una función exponencial crece tan rápidamente que relativamente al valor y del punto más alto (que determina la escala del eje y), los valores y de puntos anteriores en la curva exponencial (y todos los puntos en la curva cuadrática) son casi indistinguibles de 0.

El gráfico de la derecha aborda este problema usando una escala logarítmica en el eje y. Se puede ver fácilmente que los algoritmos exponenciales son impracticables para todo excepto las entradas más pequeñas.

Observa que cuando se grafica en una escala logarítmica, una curva exponencial aparece como una línea recta.

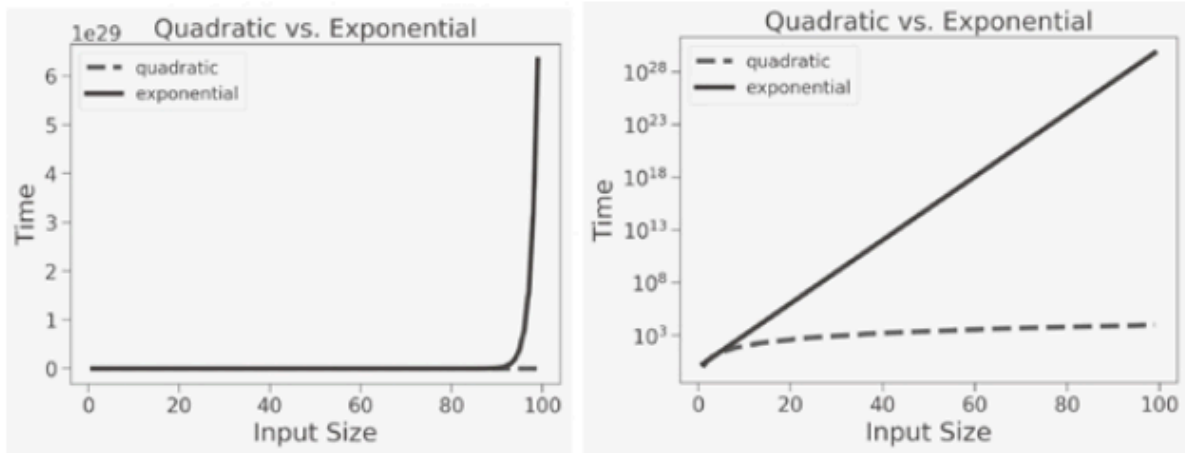


Figure 11-9 Quadratic and exponential growth

Ejercicio manual de la lectura

Pregunta 1: Simplifique $n * n + \log(n) + 2 ** a$ para determinar θ en términos de n .

- $n * n = O(n^2)$
- $\log(n) = O(\log n)$
- $2^a = O(1)$

La complejidad dominante es $O(n^2)$.

Pregunta 2: Simplifica $2 ** n + n * \log(n) + n ** 2$ para determinar θ en términos de n .

- $2^n = O(2^n)$
- $n * \log(n) = O(n \log n)$
- $n^2 = O(n^2)$

La complejidad dominante es $O(2^n)$

Pregunta 3: Simplifica $f * \log(f) + 100000 + 300 * a + x * y * z$ para determinar θ en términos de n .

Para poder determinar la complejidad necesitaríamos saber la relación de f , a , x y z con n .